Fuzz Testing in Go

Ayça Aydoğan



What is Fuzzing?



How does it work?



How to Fuzz?



Demo



Real life example



fuzzy

adjective

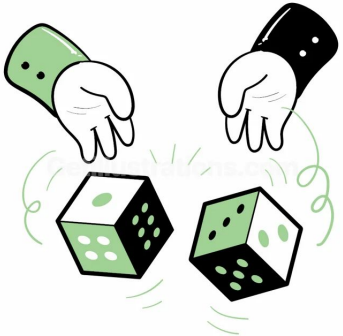
UK /ˈfʌz.i/ US /ˈfʌz.i/

fuzzy *adjective* (NOT CLEAR)

Add to word list

(of an image) having shapes that do not have clear edges, or (of a sound, especially from a television, radio, etc.) not clear, usually because of other unwanted noises making it difficult to hear:

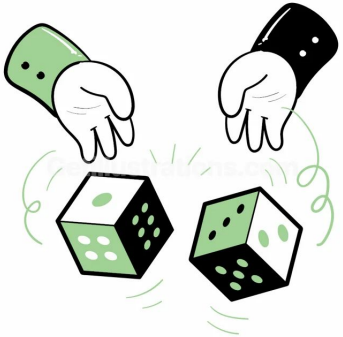
What is Fuzzing?



- A type of automated testing which continuously manipulates inputs to a program to find issues such as bugs or vulnerabilities to which the code may be susceptible.
- Professor Barton Miller, 1980s



Why Fuzzing?

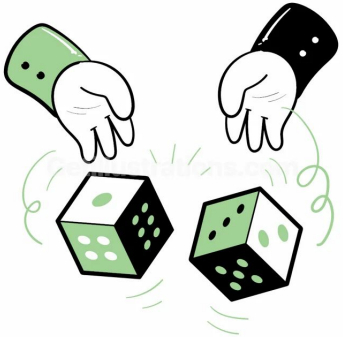


- Identify edge cases
- Unit tests are biased
- Easy to use



When to use Fuzzing?

- Parser functions
- Crypto, compression
- Codecs for audio, video, image
- Databases
- Text editor processors
- Browsers
- Formatters, template engines





Native Go Fuzzing

- Before Go 1.18
  - Packages like go-fuzz or fzgo
  - Requires more work



- With Go 1.18
  - Fuzzing got involved to the testing package

OSS Fuzz (Open Source Security Fuzz)

- Launched in 2016.
- As of August 2023, OSS-Fuzz has helped identify and fix over **10,000** vulnerabilities and **36,000** bugs across **1,000 projects.**
    - Go stdlib 200+
    - Chrome 15000+

Found bugs with native go fuzzing

Trophies

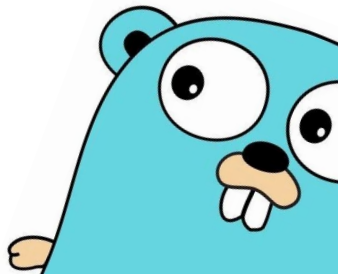
Standard Library

- go/scanner: inconsistent handling of NUL bytes in 1.17
- time: ParseDuration can panic on invalid input

Other projects

- go-yaml/yaml/v3: yaml.Unmarshal() crashes on "#\n - - Ql\xd7"
- josephburnett/jd: crashes when validating JSON Patch test/replace operation
- josephburnett/jd: JSON Patch to void failing round-trip
- kokes/smda: nil pointer error
- kokes/smda: out of bounds error
- mvdan.cc/sh: syntax.Quote failed to quote the empty string
- mvdan.cc/sh: syntax.Quote misused `\x` escape sequences in mksh
- mvdan.cc/sh: syntax.Quote can't quote the `0xFFFE` and `0xFFFF` runes in mksh
- pelletier/go-toml: index out of range error in expect function
- swaggest/form: panic on invalid keys in request
- yuin/goldmark: corner case errors
- DataDog/datadog-agent: fix edge case in tags normalization

https://github.com/golang/go/wiki/Fuzzing-trophy-case



Fuzz Types



Blackbox fuzzing



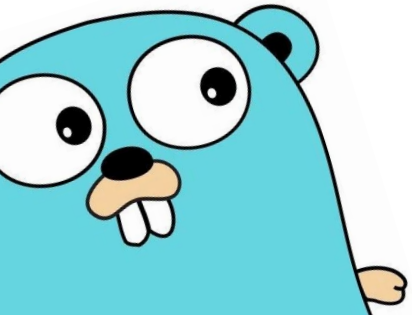
Greybox fuzzing
(Coverage guided)



Whitebox fuzzing

How it works?

Coverage-guided fuzzers maximize the amount of executed code during fuzzing. To cover new code, the fuzzer tries to mutate test cases in such a way that they pass existing checks and reach yet uncovered code paths. Coverage-guided fuzzers rely on feedback from the fuzzed application during runtime to detect if a given mutation has reached new code and should be explored further.



How it works?



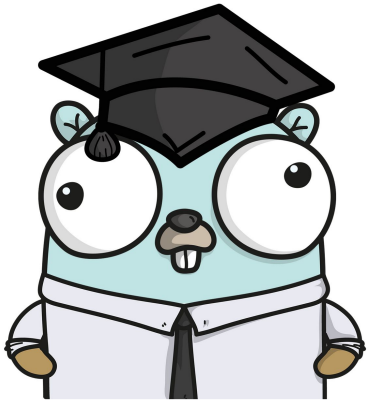
- **Seed Corpus** is the user-specified set of inputs to a fuzz test which will be run by default with go test.

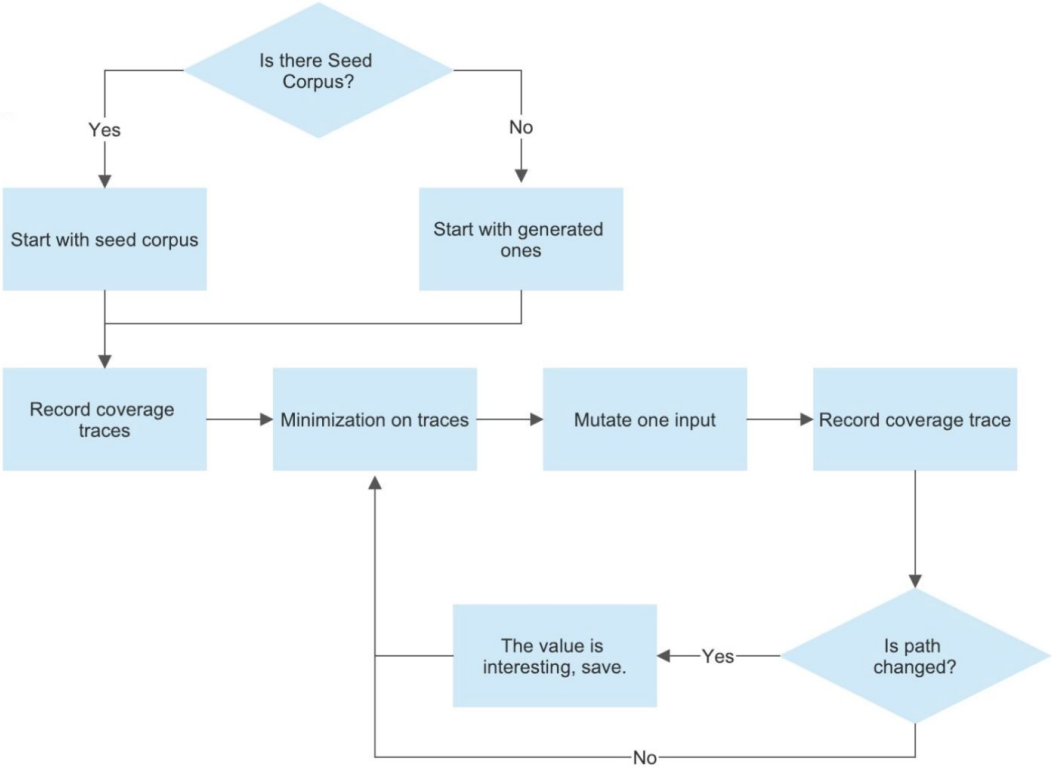
- The **mutator** works with a generator to mutate bytes to be used as input to the fuzz test.

- The **fuzzing engine** maintains the corpus, invokes the mutator, identifies new coverage, and reports failures. Also, writes the values that expand the test coverage into $GOCACHE/fuzz while running.

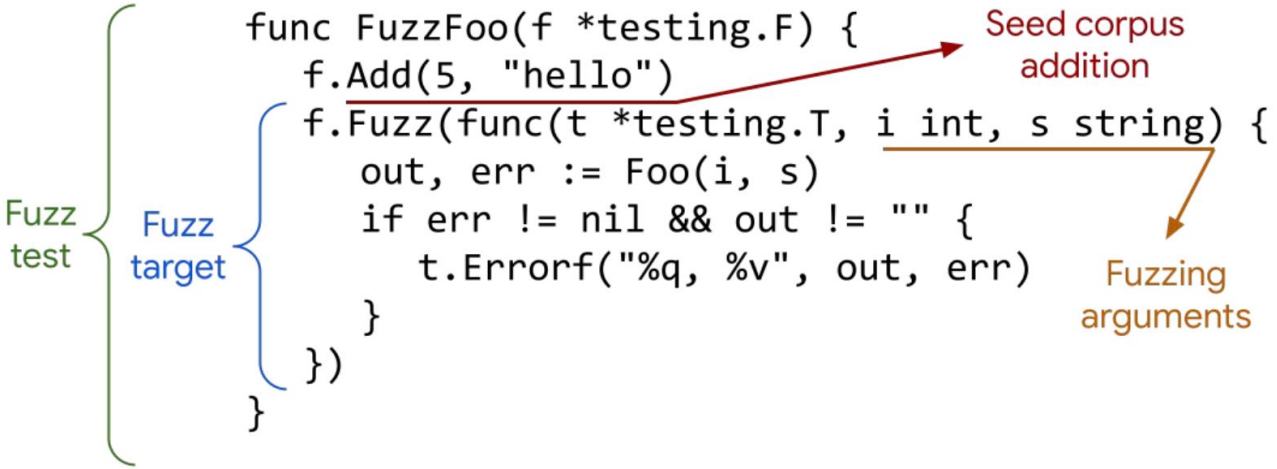
How it works?

Coverage Tracking





How to Fuzz?



https://go.dev/security/fuzz/example.png

$go test -fuzz=FuzzFoo

How to Fuzz?

- **The seed corpus** can also be given in a file in the testdata/fuzz/FuzzFoo directory instead of f.Add()

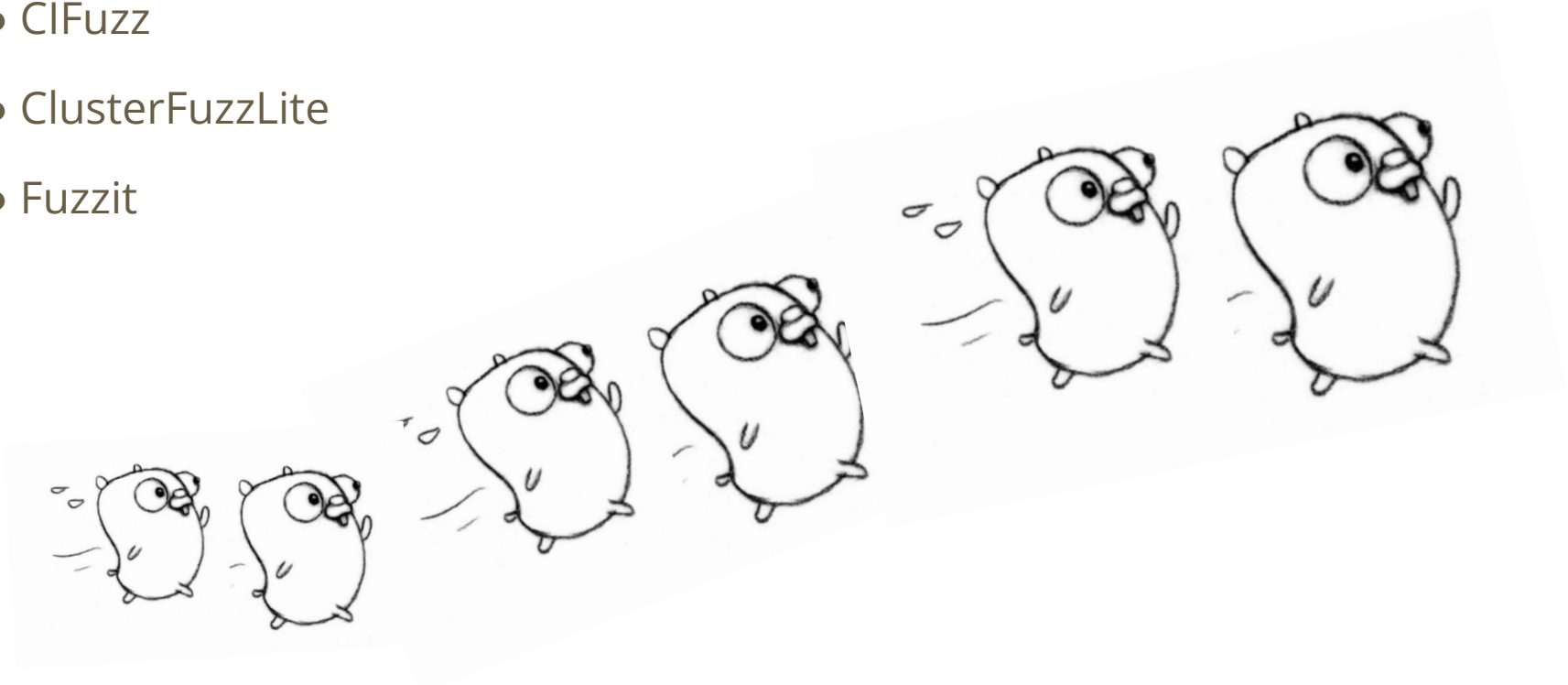
testdata > fuzz > FuzzReverse > 91862839dc552bd95b4e42be6576a6c198

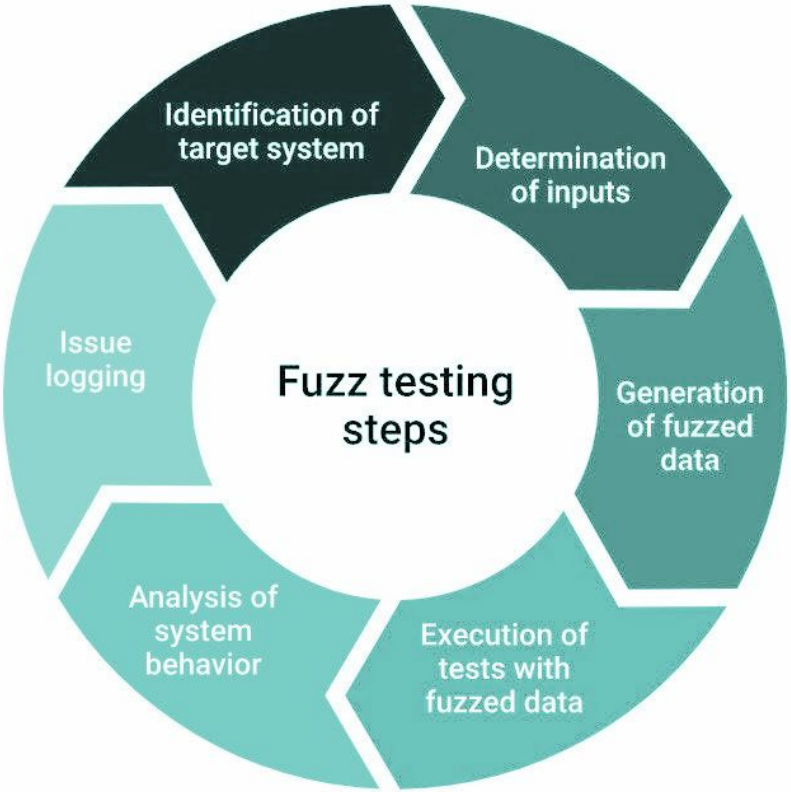
```
1   go test fuzz v1
2   string("seed input")
```

- **Fuzzing arguments** can only have the following types: string, bool, float32, float64, int, int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64, []byte

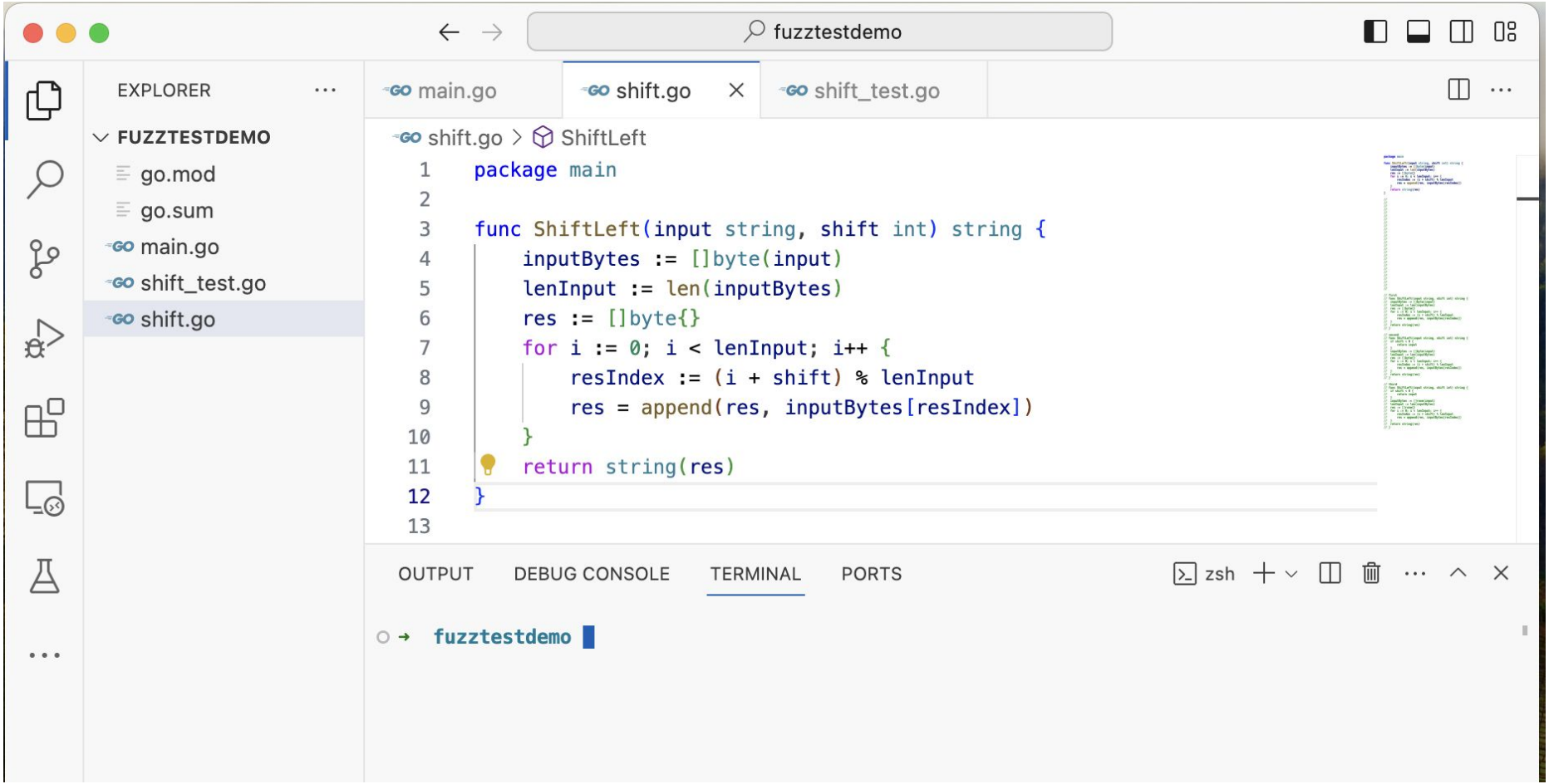
Continuous Fuzzing

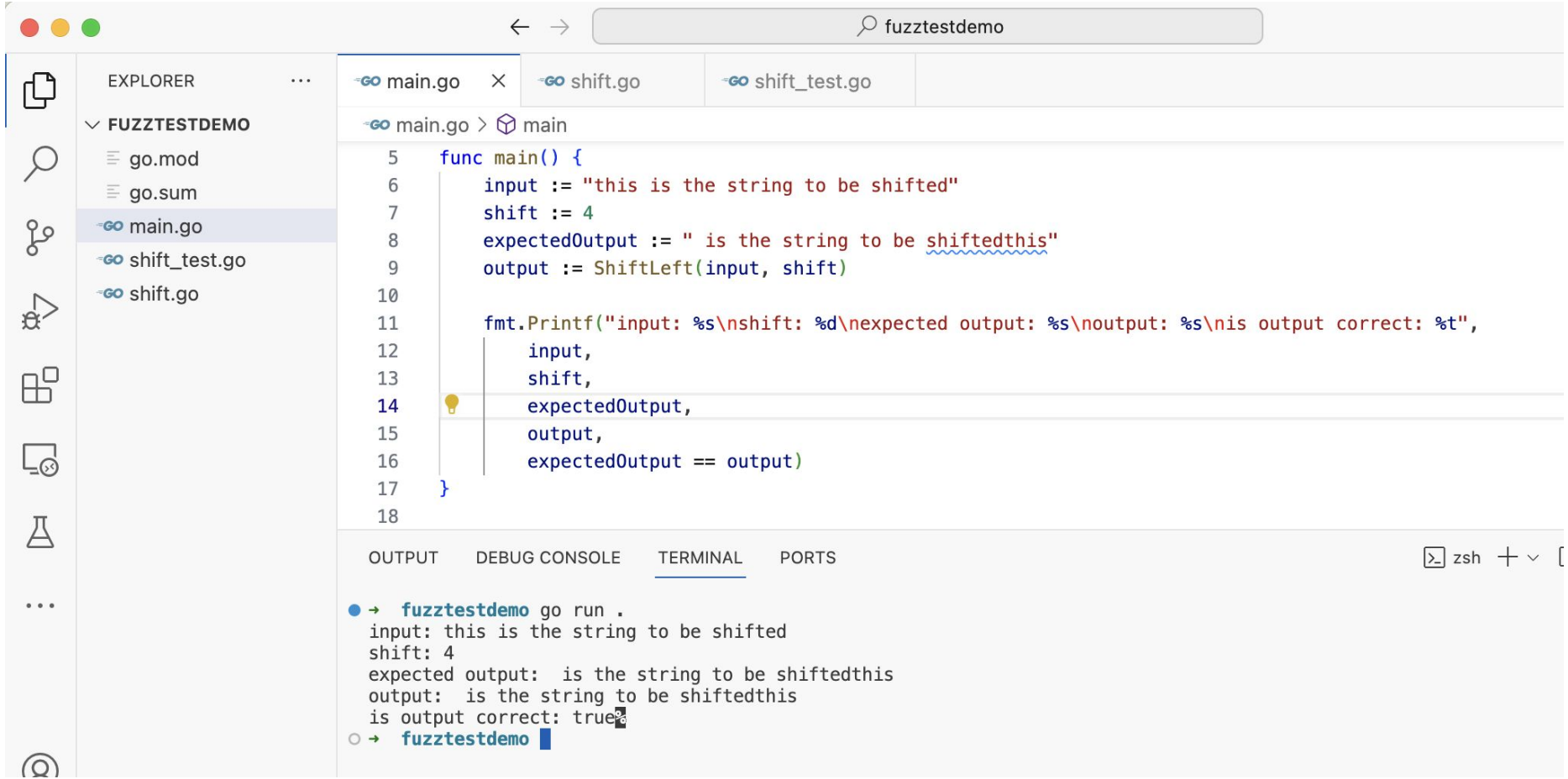
- CIFuzz
- ClusterFuzzLite
- Fuzzit




Fuzz testing steps
Identification of target system
Determination of inputs
Generation of fuzzed data
Execution of tests with fuzzed data
Analysis of system behavior
Issue logging

DEMO

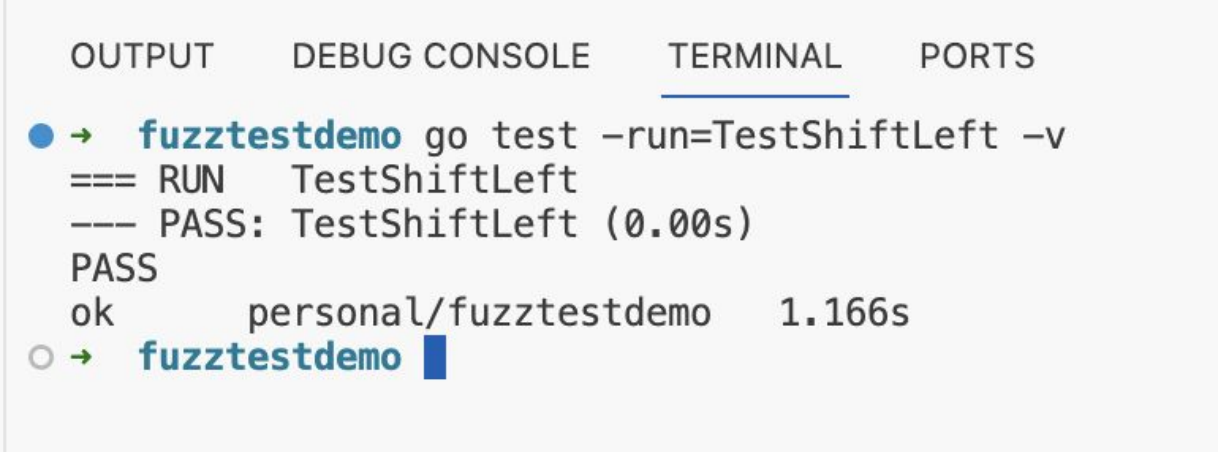

fuzztestdemo
EXPLORER
FUZZTESTDEMO
go.mod
go.sum
main.go
shift_test.go
shift.go
main.go
shift.go
shift_test.go
shift.go > ShiftLeft
1 package main
2
3 func ShiftLeft(input string, shift int) string {
4 inputBytes := []byte(input)
5 lenInput := len(inputBytes)
6 res := []byte{}
7 for i := 0; i < lenInput; i++ {
8 resIndex := (i + shift) % lenInput
9 res = append(res, inputBytes[resIndex])
10 }
11 return string(res)
12 }
13
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
fuzztestdemo

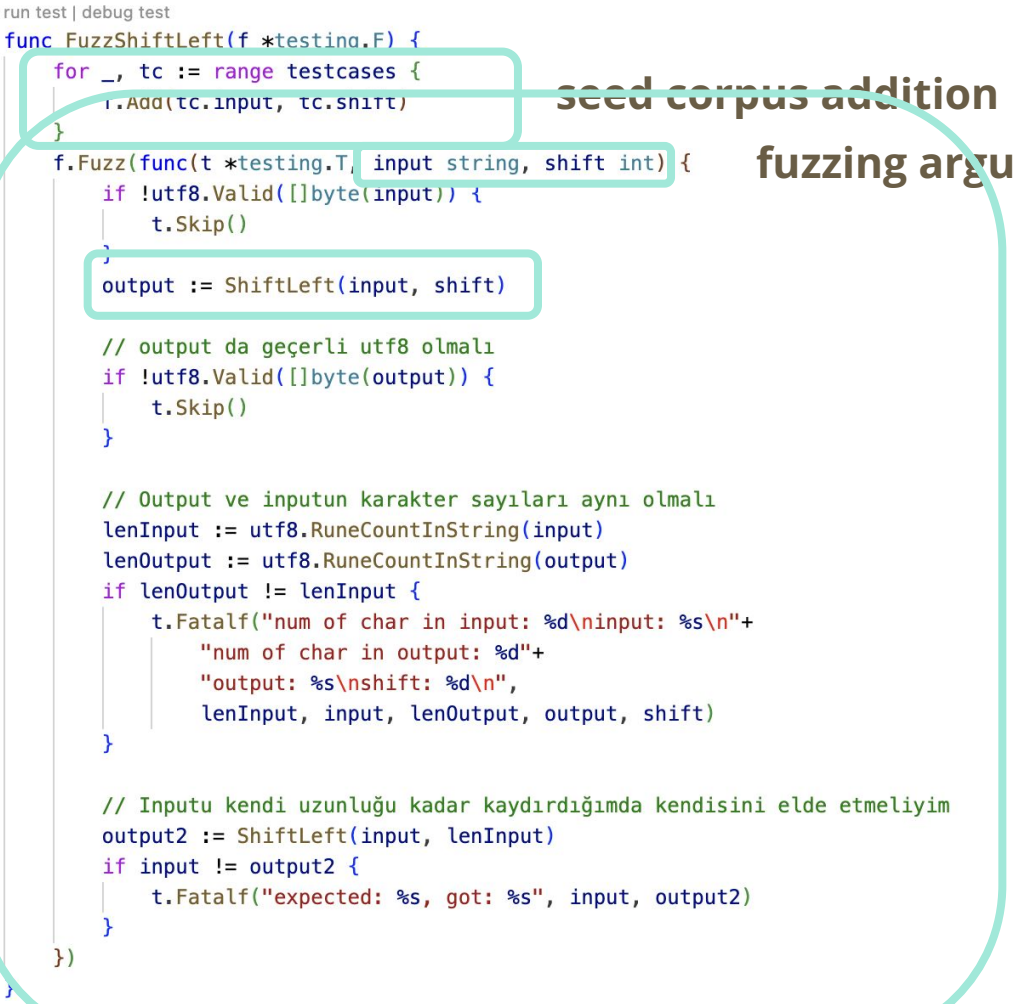

fuzztestdemo
EXPLORER
FUZZTESTDEMO
go.mod
go.sum
main.go
shift_test.go
shift.go
main.go
shift.go
shift_test.go
main.go > main
5 func main() {
6 input := "this is the string to be shifted"
7 shift := 4
8 expectedOutput := " is the string to be shiftedthis"
9 output := ShiftLeft(input, shift)
10
11 fmt.Printf("input: %s\nshift: %d\nexpected output: %s\noutput: %s\nis output correct: %t",
12 input,
13 shift,
14 expectedOutput,
15 output,
16 expectedOutput == output)
17 }
18
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
➜ fuzztestdemo go run .
input: this is the string to be shifted
shift: 4
expected output: is the string to be shiftedthis
output: is the string to be shiftedthis
is output correct: true%
➜ fuzztestdemo

```
var testcases = []struct {
    input    string
    shift    int
    expected string
}{
    {
        input:    "",
        shift:    7,
        expected: "",
    },
    {
        input:    "$%~ghff",
        shift:    0,
        expected: "$%~ghff",
    },
    {
        input:    "exa4mple!!!!",
        shift:    7,
        expected: "e!!!!exa4mpl",
    },
    {
        input:    "Hello, world!!",
        shift:    15,
        expected: "ello, world!!H",
    },
}
```

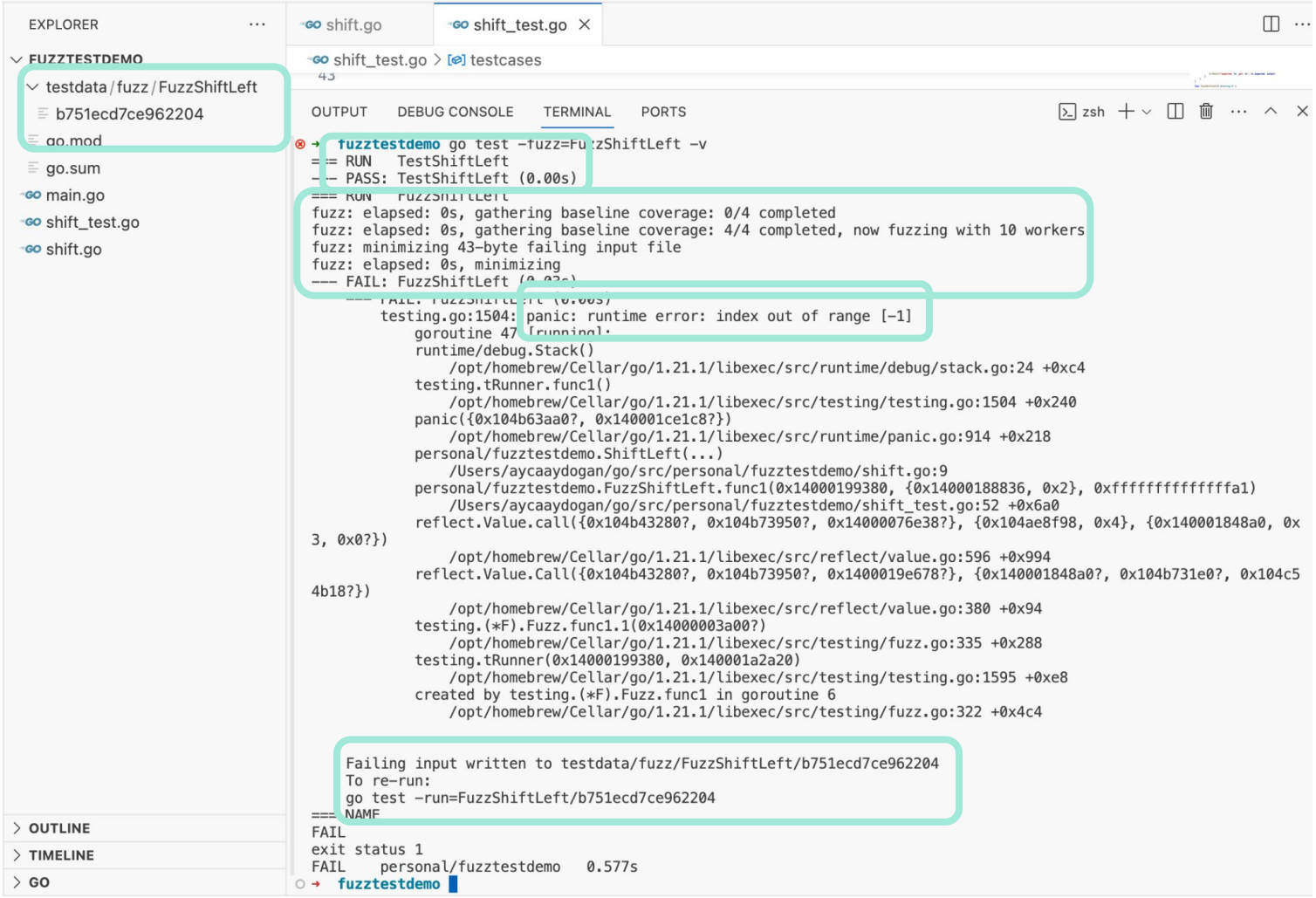
run test | debug test

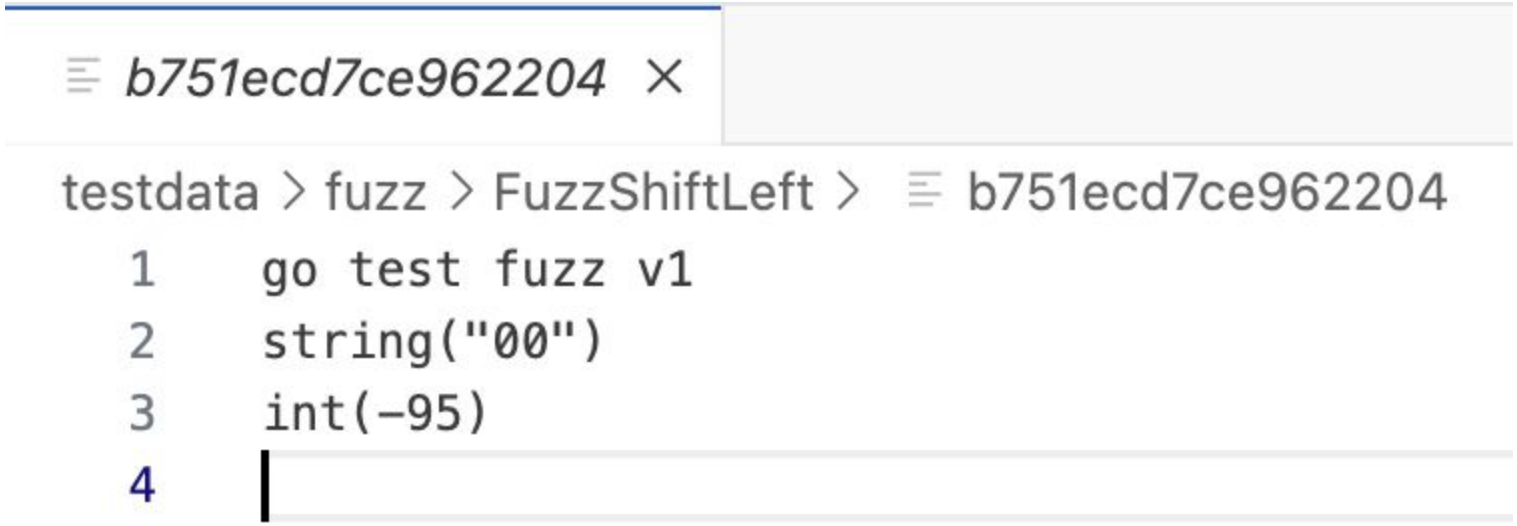
```go
func TestShiftLeft(t *testing.T) {
    for _, tc := range testcases {
        output := ShiftLeft(tc.input, tc.shift)
        if tc.expected != output {
            t.Fatalf("expected: %v, got: %v", tc.expected, output)
        }
    }
}
```




run test | debug test
func FuzzShiftLeft(f *testing.F) {
for _, tc := range testcases {
f.Add(tc.input, tc.shift)
}
f.Fuzz(func(t *testing.T, input string, shift int) {
if !utf8.Valid([]byte(input)) {
t.Skip()
}
output := ShiftLeft(input, shift)
// output da geçerli utf8 olmalı
if !utf8.Valid([]byte(output)) {
t.Skip()
}
// Output ve inputun karakter sayıları aynı olmalı
lenInput := utf8.RuneCountInString(input)
lenOutput := utf8.RuneCountInString(output)
if lenOutput != lenInput {
t.Fatalf("num of char in input: %d\ninput: %s\n"+
"num of char in output: %d"+
"output: %s\nshift: %d\n",
lenInput, input, lenOutput, output, shift)
}
// Inputu kendi uzunluğu kadar kaydırdığımda kendisini elde etmeliyim
output2 := ShiftLeft(input, lenInput)
if input != output2 {
t.Fatalf("expected: %s, got: %s", input, output2)
}
})
}
seed corpus addition
fuzzing arguments
fuzz target

```
● → fuzztestdemo go test -run=FuzzShiftLeft -v
=== RUN   FuzzShiftLeft
=== RUN   FuzzShiftLeft/seed#0
=== RUN   FuzzShiftLeft/seed#1
=== RUN   FuzzShiftLeft/seed#2
=== RUN   FuzzShiftLeft/seed#3
--- PASS: FuzzShiftLeft (0.00s)
    --- PASS: FuzzShiftLeft/seed#0 (0.00s)
    --- PASS: FuzzShiftLeft/seed#1 (0.00s)
    --- PASS: FuzzShiftLeft/seed#2 (0.00s)
    --- PASS: FuzzShiftLeft/seed#3 (0.00s)
PASS
ok      personal/fuzztestdemo   1.249s
○ → fuzztestdemo
```


EXPLORER
FUZZTESTDEMO
testdata / fuzz / FuzzShiftLeft
b751ecd7ce962204
go.mod
go.sum
main.go
shift_test.go
shift.go
OUTLINE
TIMELINE
GO
shift.go
shift_test.go
shift_test.go > testcases
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
→ fuzztestdemo go test -fuzz=FuzzShiftLeft -v
=== RUN TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/4 completed
fuzz: elapsed: 0s, gathering baseline coverage: 4/4 completed, now fuzzing with 10 workers
fuzz: minimizing 43-byte failing input file
fuzz: elapsed: 0s, minimizing
--- FAIL: FuzzShiftLeft (0.03s)
testing.go:1504: panic: runtime error: index out of range [-1]
goroutine 47 [running]:
runtime/debug.Stack()
/opt/homebrew/Cellar/go/1.21.1/libexec/src/runtime/debug/stack.go:24 +0xc4
testing.tRunner.func1()
/opt/homebrew/Cellar/go/1.21.1/libexec/src/testing/testing.go:1504 +0x240
panic({0x104b63aa0?, 0x140001ce1c8?})
/opt/homebrew/Cellar/go/1.21.1/libexec/src/runtime/panic.go:914 +0x218
personal/fuzztestdemo.ShiftLeft(...)
/Users/aycaaydogan/go/src/personal/fuzztestdemo/shift.go:9
personal/fuzztestdemo.FuzzShiftLeft.func1(0x14000199380, {0x14000188836, 0x2}, 0xffffffffffffffa1)
/Users/aycaaydogan/go/src/personal/fuzztestdemo/shift_test.go:52 +0x6a0
reflect.Value.call({0x104b43280?, 0x104b73950?, 0x14000076e38?}, {0x104ae8f98, 0x4}, {0x140001848a0, 0x3, 0x0?})
/opt/homebrew/Cellar/go/1.21.1/libexec/src/reflect/value.go:596 +0x994
reflect.Value.Call({0x104b43280?, 0x104b73950?, 0x1400019e678?}, {0x140001848a0?, 0x104b731e0?, 0x104c54b18?})
/opt/homebrew/Cellar/go/1.21.1/libexec/src/reflect/value.go:380 +0x94
testing.(*F).Fuzz.func1.1(0x14000003a00?)
/opt/homebrew/Cellar/go/1.21.1/libexec/src/testing/fuzz.go:335 +0x288
testing.tRunner(0x14000199380, 0x140001a2a20)
/opt/homebrew/Cellar/go/1.21.1/libexec/src/testing/testing.go:1595 +0xe8
created by testing.(*F).Fuzz.func1 in goroutine 6
/opt/homebrew/Cellar/go/1.21.1/libexec/src/testing/fuzz.go:322 +0x4c4
Failing input written to testdata/fuzz/FuzzShiftLeft/b751ecd7ce962204
To re-run:
go test -run=FuzzShiftLeft/b751ecd7ce962204
=== NAME
FAIL
exit status 1
FAIL personal/fuzztestdemo 0.577s
→ fuzztestdemo


b751ecd7ce962204
testdata > fuzz > FuzzShiftLeft > b751ecd7ce962204
1 go test fuzz v1
2 string("00")
3 int(-95)
4

shift.go

shift.go > ShiftLeft

```go
package main

func ShiftLeft(input string, shift int) string {
    inputBytes := []byte(input)
    lenInput := len(inputBytes)
    res := []byte{}
    for i := 0; i < lenInput; i++ {
        resIndex := (i + shift) % lenInput
        res = append(res, inputBytes[resIndex])
    }
    return string(res)
}

```

shift.go

shift.go > ...

```go
package main

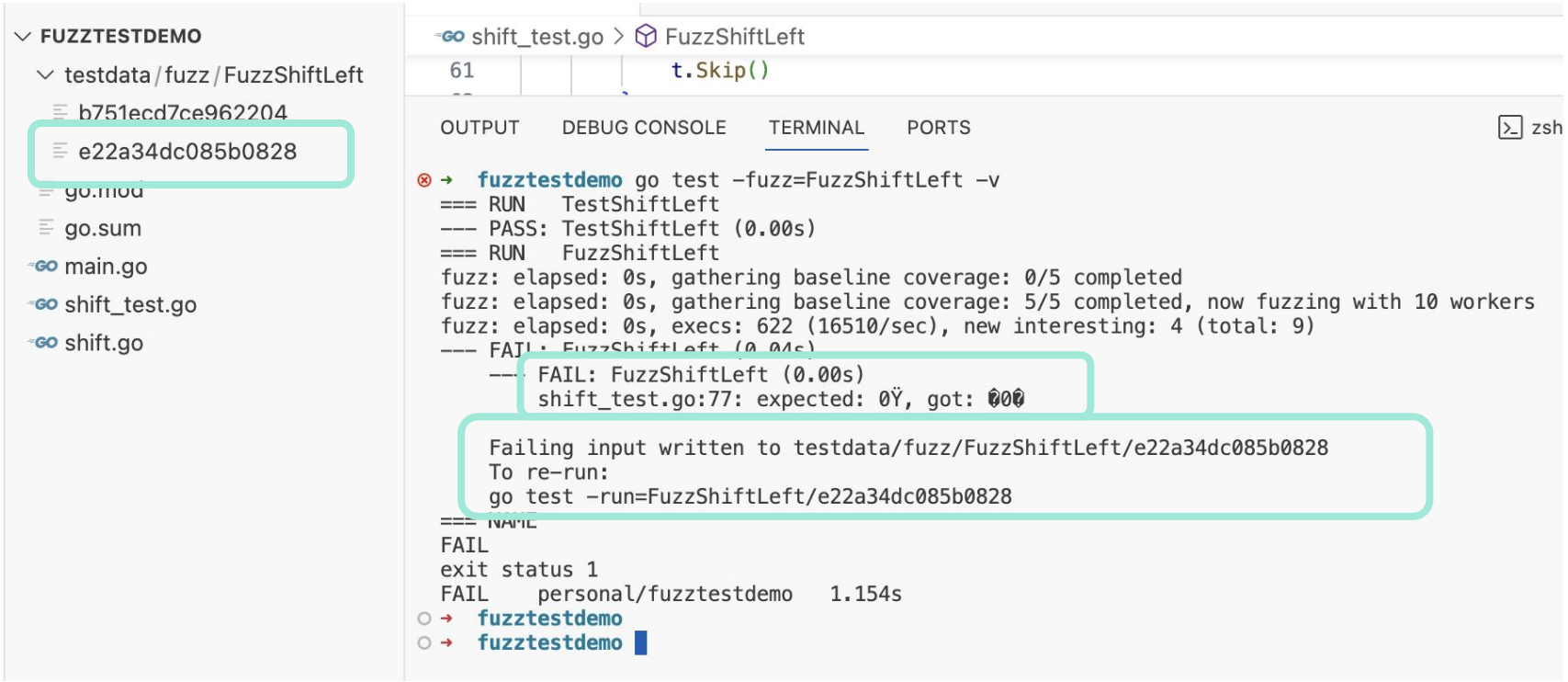
func ShiftLeft(input string, shift int) string {
    if shift < 0 {
        return input
    }
    inputBytes := []byte(input)
    lenInput := len(inputBytes)
    res := []byte{}
    for i := 0; i < lenInput; i++ {
        resIndex := (i + shift) % lenInput
        res = append(res, inputBytes[resIndex])
    }
    return string(res)
}
```

```
→  fuzztestdemo go test -run=FuzzShiftLeft/b751ecd7ce962204
PASS
ok      personal/fuzztestdemo   1.185s
```

```
33      {
34          input:    "00",
35          shift:    -95,
36          expected: "00",
37      },
38  }
39
    run test | debug test
40  func TestShiftLeft(t *testing.T) {
41      for _, tc := range testcases {
42          output := ShiftLeft(tc.input, tc.shift)
43          if tc.expected != output {
44              t.Fatalf("expected: %v, got: %v", tc.expected, output)
45          }
46      }
47  }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
→  fuzztestdemo go test -run=TestShiftLeft -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
PASS
ok      personal/fuzztestdemo   0.457s
```


FUZZTESTDEMO
testdata / fuzz / FuzzShiftLeft
b751ecd7ce962204
e22a34dc085b0828
go.mod
go.sum
main.go
shift_test.go
shift.go
shift_test.go > FuzzShiftLeft
61 t.Skip()
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
zsh
fuzztestdemo go test -fuzz=FuzzShiftLeft -v
=== RUN TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/5 completed
fuzz: elapsed: 0s, gathering baseline coverage: 5/5 completed, now fuzzing with 10 workers
fuzz: elapsed: 0s, execs: 622 (16510/sec), new interesting: 4 (total: 9)
FAIL: FuzzShiftLeft (0.00s)
shift_test.go:77: expected: 0Ÿ, got: ��
Failing input written to testdata/fuzz/FuzzShiftLeft/e22a34dc085b0828
To re-run:
go test -run=FuzzShiftLeft/e22a34dc085b0828
FAIL
exit status 1
FAIL personal/fuzztestdemo 1.154s
fuzztestdemo
fuzztestdemo

```
run test | debug test
func FuzzShiftLeft(f *testing.F) {
    for _, tc := range testcases {
        f.Add(tc.input, tc.shift)
    }
    f.Fuzz(func(t *testing.T, input string, shift int) {
        if !utf8.Valid([]byte(input)) {
            t.Skip()
        }
        output := ShiftLeft(input, shift)

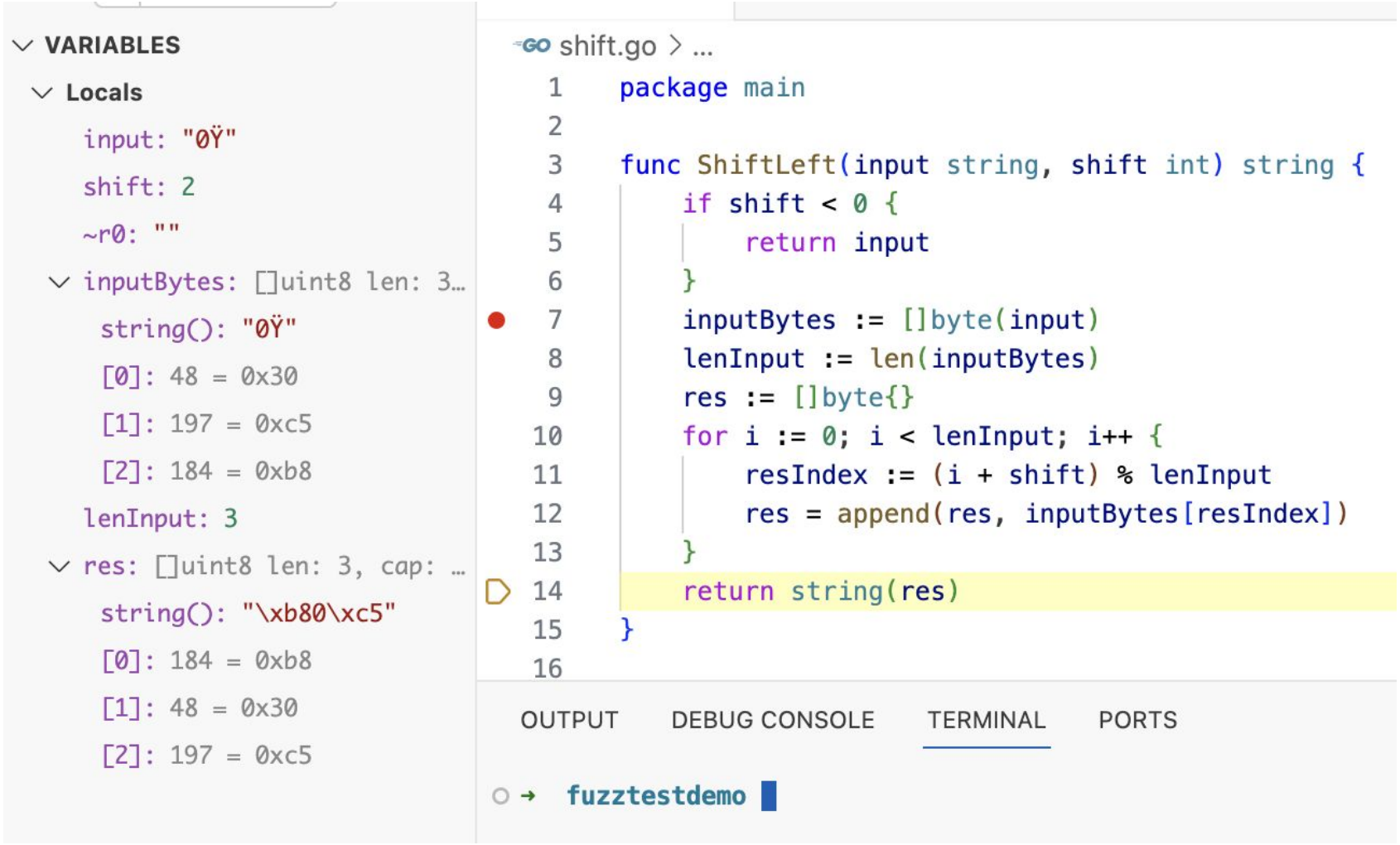
        // output da geçerli utf8 olmalı
        if !utf8.Valid([]byte(output)) {
            t.Skip()
        }

        // Output ve inputun karakter sayıları aynı olmalı
        lenInput := utf8.RuneCountInString(input)
        lenOutput := utf8.RuneCountInString(output)
        if lenOutput != lenInput {
            t.Fatalf("num of char in input: %d\ninput: %s\n"+
                "num of char in output: %d"+
                "output: %s\nshift: %d\n",
                lenInput, input, lenOutput, output, shift)
        }

        // Inputu kendi uzunluğu kadar kaydırdığımda kendisini elde etmeliyim
        output2 := ShiftLeft(input, lenInput)
        if input != output2 {
            t.Fatalf("expected: %s, got: %s", input, output2)
        }
    })
}

```


e22a34dc085b0828
testdata > fuzz > FuzzShiftLeft > e22a34dc085b0828
1 go test fuzz v1
2 string("0Ÿ")
3 int(7)
4


VARIABLES
Locals
input: "0Ÿ"
shift: 2
~r0: ""
inputBytes: []uint8 len: 3…
string(): "0Ÿ"
[0]: 48 = 0x30
[1]: 197 = 0xc5
[2]: 184 = 0xb8
lenInput: 3
res: []uint8 len: 3, cap: …
string(): "\xb80\xc5"
[0]: 184 = 0xb8
[1]: 48 = 0x30
[2]: 197 = 0xc5
shift.go > ...
1 package main
2
3 func ShiftLeft(input string, shift int) string {
4 if shift < 0 {
5 return input
6 }
7 inputBytes := []byte(input)
8 lenInput := len(inputBytes)
9 res := []byte{}
10 for i := 0; i < lenInput; i++ {
11 resIndex := (i + shift) % lenInput
12 res = append(res, inputBytes[resIndex])
13 }
14 return string(res)
15 }
16
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
fuzztestdemo

```go
package main

func ShiftLeft(input string, shift int) string {
    if shift < 0 {
        return input
    }
    inputBytes := []rune(input)
    lenInput := len(inputBytes)
    res := []rune{}
    for i := 0; i < lenInput; i++ {
        resIndex := (i + shift) % lenInput
        res = append(res, inputBytes[resIndex])
    }
    return string(res)
}
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
→ fuzztestdemo go test -run=TestShiftLeft/e22a34dc085b0828 -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
PASS
ok      personal/fuzztestdemo   1.165s
```

shift_test.go > FuzzShiftLeft

```
37      },
38      {
39          input:    "0Ÿ",
40          shift:    7,
41          expected: "Ÿ0",
42      },
43  }
44
    run test | debug test
45  func TestShiftLeft(t *testing.T) {
46      for _, tc := range testcases {
47          output := ShiftLeft(tc.input, tc.shift)
48          if tc.expected != output {
49              t.Fatalf("expected: %v, got: %v", tc.expected, output)
50          }
51      }
52  }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
→  fuzztestdemo go test -run=TestShiftLeft -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
PASS
ok      personal/fuzztestdemo   1.096s
```

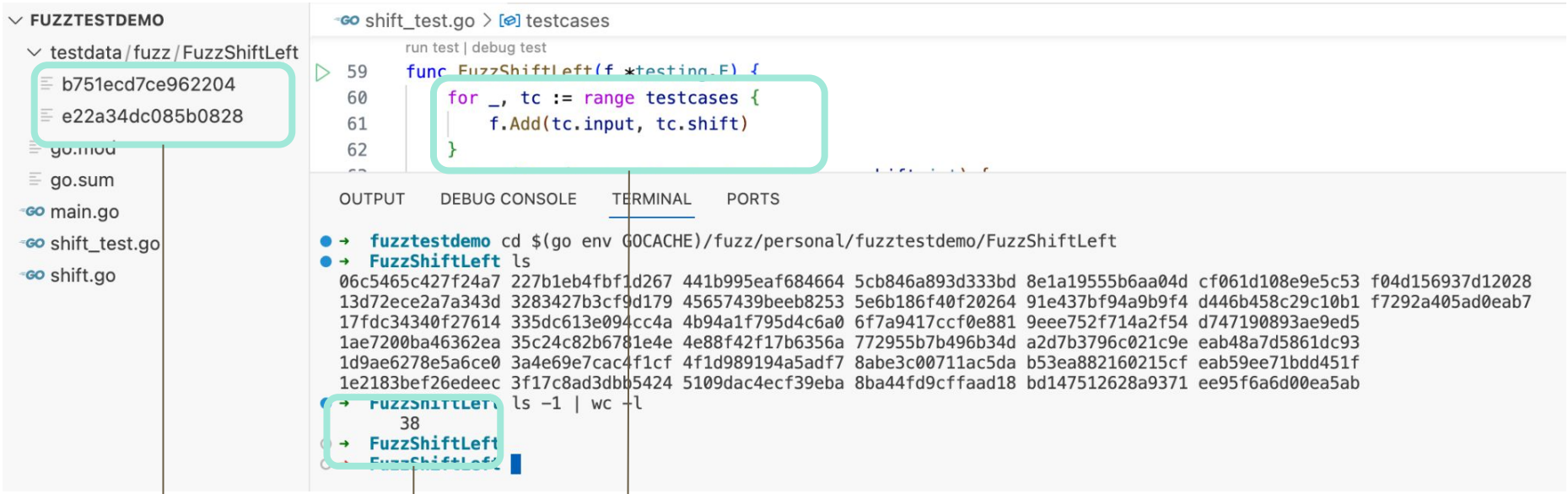
shift_test.go > FuzzShiftLeft

```
43      {
44          input:    "0Ÿ",
45          shift:    2,
46          expected: "0Ÿ",
47      },
48  }
49
    run test | debug test
50  func TestShiftLeft(t *testing.T) {
51      for _, tc := range testcases {
52          output := ShiftLeft(tc.input, tc.shift)
53          if tc.expected != output {
54              t.Fatalf("expected: %v, got: %v", tc.expected, output)
55          }
56      }
57  }
```

OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
→  fuzztestdemo go test -run=TestShiftLeft -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
PASS
ok      personal/fuzztestdemo   0.459s
```

```
➜  fuzztestdemo go test -fuzz=FuzzShiftLeft -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN   FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/11 completed
fuzz: elapsed: 0s, gathering baseline coverage: 11/11 completed, now fuzzing with 10 workers
fuzz: elapsed: 3s, execs: 568703 (189503/sec), new interesting: 28 (total: 39)
fuzz: elapsed: 6s, execs: 1664905 (365519/sec), new interesting: 30 (total: 41)
fuzz: elapsed: 9s, execs: 3394342 (576490/sec), new interesting: 31 (total: 42)
fuzz: elapsed: 12s, execs: 4654907 (420173/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 15s, execs: 5891510 (412201/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 18s, execs: 7140693 (416403/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 21s, execs: 8382803 (413935/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 24s, execs: 9635082 (417533/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 27s, execs: 10903566 (422826/sec), new interesting: 32 (total: 43)
fuzz: elapsed: 30s, execs: 12141902 (412641/sec), new interesting: 33 (total: 44)
fuzz: elapsed: 33s, execs: 13401940 (420029/sec), new interesting: 33 (total: 44)
fuzz: elapsed: 36s, execs: 14656132 (418179/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 39s, execs: 15884837 (409568/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 42s, execs: 17133614 (416188/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 45s, execs: 18380813 (415793/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 48s, execs: 19611301 (410055/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 51s, execs: 20864333 (417777/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 54s, execs: 22076597 (403983/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 57s, execs: 23346793 (423399/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 1m0s, execs: 24605415 (419542/sec), new interesting: 34 (total: 45)
fuzz: elapsed: 1m3s, execs: 25859310 (418092/sec), new interesting: 34 (total: 45)
^Cfuzz: elapsed: 1m5s, execs: 26893429 (416547/sec), new interesting: 34 (total: 45)
--- PASS: FuzzShiftLeft (65.49s)
=== NAME
PASS
ok      personal/fuzztestdemo   66.668s
➜  fuzztestdemo
```


FUZZTESTDEMO
testdata / fuzz / FuzzShiftLeft
b751ecd7ce962204
e22a34dc085b0828
go.sum
main.go
shift_test.go
shift.go
shift_test.go > testcases
run test | debug test
59 func FuzzShiftLeft(f *testing.F) {
60 for _, tc := range testcases {
61 f.Add(tc.input, tc.shift)
62 }
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
fuzztestdemo cd $(go env GOCACHE)/fuzz/personal/fuzztestdemo/FuzzShiftLeft
FuzzShiftLeft ls
06c5465c427f24a7 227b1eb4fbf1d267 441b995eaf684664 5cb846a893d333bd 8e1a19555b6aa04d cf061d108e9e5c53 f04d156937d12028
13d72ece2a7a343d 3283427b3cf9d179 45657439beeb8253 5e6b186f40f20264 91e437bf94a9b9f4 d446b458c29c10b1 f7292a405ad0eab7
17fdc34340f27614 335dc613e094cc4a 4b94a1f795d4c6a0 6f7a9417ccf0e881 9eee752f714a2f54 d747190893ae9ed5
1ae7200ba46362ea 35c24c82b6781e4e 4e88f42f17b6356a 772955b7b496b34d a2d7b3796c021c9e eab48a7d5861dc93
1d9ae6278e5a6ce0 3a4e69e7cac4f1cf 4f1d989194a5adf7 8abe3c00711ac5da b53ea882160215cf eab59ee71bdd451f
1e2183bef26edeec 3f17c8ad3dbb5424 5109dac4ecf39eba 8ba44fd9cffaad18 bd147512628a9371 ee95f6a6d00ea5ab
ls -1 | wc -l
38
FuzzShiftLeft

2 + 38 + 4+1 = 45

```
● ➜  FuzzShiftLeft vim 5109dac4ecf39eba
```

```
go test fuzz v1
string("\x9c")
int(7)
~
```

```
⊗ ➜  fuzztestdemo go test -fuzz=FuzzShiftLeft -v
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN   FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/45 completed
fuzz: elapsed: 0s, gathering baseline coverage: 45/45 completed, now fuzzing with 10 workers
fuzz: elapsed: 3s, execs: 1215044 (404981/sec), new interesting: 0 (total: 45)
fuzz: elapsed: 6s, execs: 2479641 (421538/sec), new interesting: 0 (total: 45)
fuzz: elapsed: 9s, execs: 3748764 (423061/sec), new interesting: 1 (total: 46)
fuzz: elapsed: 12s, execs: 5016868 (422588/sec), new interesting: 1 (total: 46)
fuzz: elapsed: 15s, execs: 6279224 (420904/sec), new interesting: 1 (total: 46)
fuzz: elapsed: 18s, execs: 7534339 (418374/sec), new interesting: 1 (total: 46)
^Cfuzz: elapsed: 20s, execs: 8309202 (416810/sec), new interesting: 1 (total: 46)
--- PASS: FuzzShiftLeft (19.87s)
=== NAME
PASS
ok      personal/fuzztestdemo   20.991s
○ ➜  fuzztestdemo
```

```
● → fuzztestdemo go test -fuzz=FuzzShiftLeft -v -fuzztime=10s
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN   FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/46 completed
fuzz: elapsed: 0s, gathering baseline coverage: 46/46 completed, now fuzzing with 10 workers
fuzz: elapsed: 3s, execs: 1226833 (408855/sec), new interesting: 0 (total: 46)
fuzz: elapsed: 6s, execs: 2483763 (418990/sec), new interesting: 0 (total: 46)
fuzz: elapsed: 9s, execs: 3736587 (417543/sec), new interesting: 0 (total: 46)
fuzz: elapsed: 10s, execs: 4168793 (393940/sec), new interesting: 0 (total: 46)
--- PASS: FuzzShiftLeft (10.10s)
=== NAME
PASS
ok      personal/fuzztestdemo   10.506s
○ → fuzztestdemo
```

```
→  fuzztestdemo go test -fuzz=FuzzShiftLeft -v -fuzztime=10s -parallel=3
=== RUN   TestShiftLeft
--- PASS: TestShiftLeft (0.00s)
=== RUN   FuzzShiftLeft
fuzz: elapsed: 0s, gathering baseline coverage: 0/46 completed
fuzz: elapsed: 0s, gathering baseline coverage: 46/46 completed, now fuzzing with 3 workers
fuzz: elapsed: 3s, execs: 426152 (142002/sec), new interesting: 0 (total: 46)
fuzz: elapsed: 6s, execs: 865999 (146664/sec), new interesting: 1 (total: 47)
fuzz: elapsed: 9s, execs: 1305302 (146425/sec), new interesting: 1 (total: 47)
fuzz: elapsed: 10s, execs: 1453346 (137883/sec), new interesting: 1 (total: 47)
--- PASS: FuzzShiftLeft (10.08s)
=== NAME
PASS
ok      personal/fuzztestdemo   11.147s
→  fuzztestdemo
```

Real life example

Trophies

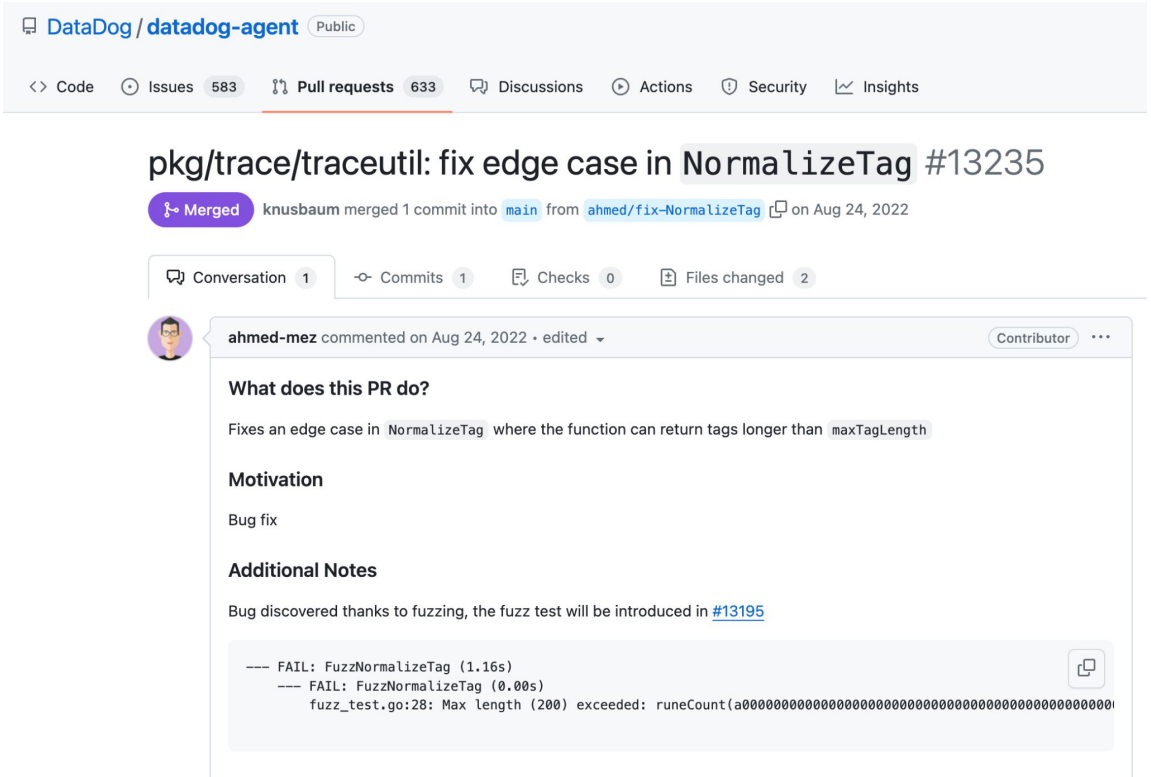
Standard Library

- go/scanner: inconsistent handling of NUL bytes in 1.17
- time: ParseDuration can panic on invalid input

Other projects

- go-yaml/yaml/v3: yaml.Unmarshal() crashes on “#\n - - Ql\xd7”
- josephburnett/jd: crashes when validating JSON Patch test/replace operation
- josephburnett/jd: JSON Patch to void failing round-trip
- kokes/smda: nil pointer error
- kokes/smda: out of bounds error
- mvdan.cc/sh: syntax.Quote failed to quote the empty string
- mvdan.cc/sh: syntax.Quote misused `\x` escape sequences in mksh
- mvdan.cc/sh: syntax.Quote can’t quote the `0xFFFE` and `0xFFFF` runes in mksh
- pelletier/go-toml: index out of range error in expect function
- swaggest/form: panic on invalid keys in request
- yuin/goldmark: corner case errors
- DataDog/datadog-agent: fix edge case in tags normalization

Real life example - PR



Real life example - fuzz test

datadog-agent / pkg / trace / traceutil / **fuzz_test.go** ↑ Top

Code | Blame | 82 lines (71 loc) · 2.81 KB | Raw

```
10  import (
11      "testing"
12      "unicode/utf8"
13  )
14
15  var normalizationSeedCorpus = []string{
16      "key:val",
17      "Data🐶dog🐶 繁がっ⛰てて",
18      "Test Conversion Of Weird !@#$%^&**() Characters",
19      "test\x99\x8faaa",
20      "A0000000000
```

Real life example - fuzz test

```
func fuzzNormalization(f *testing.F, seedCorpus []string, maxLen int, normalize func(in, lang string)
    for _, sc := range seedCorpus {
        f.Add(sc, "lang")
    }
    f.Fuzz(func(t *testing.T, input, lang string) {
        normalized, err := normalize(input, lang)
        if err != nil {
            t.Skipf("Couldn't normalize input (%s): %v", input, err)
        }
        normalizedTwice, err := normalize(normalized, lang)
        if err != nil {
            t.Fatalf("Normalizing a normalized input returned an error: %v", err)
        }
        if normalizedTwice != normalized {
            t.Fatalf("Normalizing a normalized input didn't return the same input: expected (%s) got
        }
        runeCount := utf8.RuneCountInString(normalized)
        if runeCount > maxLen {
            t.Fatalf("Max length (%d) exceeded: runeCount(%s) == %d", maxLen, normalized, runeCount)
        }
    })
}
```

Real life example - fix commit

1 pkg/trace/traceutil/normalize.go

```
@@ -154,6 +154,7 @@ func NormalizeTag(v string) string {
154 154             chars++
155 155         default:
156 156             // illegal character
    157 +           chars++
157 158             if n := len(cuts); n > 0 && cuts[n-1][1] >= i {
158 159                 // merge intersecting cuts
159 160                 cuts[n-1][1] += jump
```

4 pkg/trace/traceutil/normalize_test.go

```
@@ -67,6 +67,10 @@ func TestNormalizeTag(t *testing.T) {
67 67          {"a" + string(unicode.ReplacementChar), "a"},
68 68          {"a" + string(unicode.ReplacementChar) + string(unicode.ReplacementChar), "a"},
69 69          {"a" + string(unicode.ReplacementChar) + string(unicode.ReplacementChar) + "b", "a_b"},
   70 +        {
   71 +            in:
   "A000000000
```

Conclusion

- Complementary to unit tests
- Useful to detect bugs that are hard to predict
- Not deterministic
- No control over inputs so failures can be detected based on errors, panics or a property of the return value
- Don’t forget to integrate the fuzzed data to regression tests

References

- https://about.gitlab.com/topics/devsecops/what-is-fuzz-testing/
- https://go.dev/wiki/Fuzzing-trophy-case
- https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/main-may10.pdf
- https://www.fuzzingbook.org/html/GreyboxFuzzer.html
- https://go.dev/doc/security/fuzz/
- https://go.dev/doc/tutorial/fuzz
- https://go.googlesource.com/proposal/+/master/design/draft-fuzzing.md
- https://google.github.io/oss-fuzz/
- https://talks.osinet.fr/fuzzing-en.slide#6
- https://google.github.io/clusterfuzz/reference/coverage-guided-vs-blackbox/
- https://www.code-intelligence.com/blog/golang-fuzzing-1.19
- https://google.github.io/oss-fuzz/getting-started/continuous-integration/
- https://go.dev/src/internal/fuzz/minimize.go
- https://github.com/DataDog/datadog-agent



Thank you